

TD 6 — Contribuer à un projet open source — Corrigé

Exercice 1 — Évaluer un projet (20 min)

Exemple d'évaluation : Django

Projet choisi : <https://github.com/django/django>

Grille d'évaluation (exemple)

Critère	Valeur	Commentaire
Dernière activité	< 24h	Projet très actif
Nombre de contributeurs	> 2000	Large communauté
Contributeurs actifs	> 50	Nombreux contributeurs récents
Issues ouvertes	1500	Normal pour un projet de cette taille
PRs ouvertes	200	Traitement régulier
Présence de CONTRIBUTING.md	Oui	Documentation complète
Présence de CODE_OF_CONDUCT.md	Oui	Contributor Covenant
Labels "good first issue"	30-50	Bien maintenu
Temps moyen de merge des PRs	1-2 semaines	Raisonnable

Questions

1. Ce projet est-il "sain" pour un nouveau contributeur ? Justifiez.

Oui, Django est un projet sain pour les nouveaux contributeurs :

- **Activité régulière** : commits quotidiens, issues traitées rapidement
- **Documentation contribution** : CONTRIBUTING.md détaillé avec processus clair
- **Labels pour débutants** : "easy pickings" pour les premières contributions
- **Communauté accueillante** : Code of Conduct, forums actifs, mentoring
- **Process établi** : workflow documenté, CI/CD en place

2. Quels sont les points forts et les points faibles pour un débutant ?

Points forts :

- Documentation exhaustive (contributing guide, coding style)
- Issues bien catégorisées avec labels clairs
- Communauté active et bienveillante
- Tests automatisés qui valident les contributions
- Historique de mentoring des nouveaux contributeurs

Points faibles :

- Codebase volumineux (courbe d'apprentissage)
- Standards de qualité élevés (peut être intimidant)
- Processus de review rigoureux (plusieurs itérations nécessaires)
- Nécessite une bonne connaissance de Python et du web

3. Recommanderiez-vous ce projet pour une première contribution ?

Oui, avec nuances :

- **Recommandé si** : vous connaissez Python et Django, vous êtes patient pour le processus de review

- **Commencer par** : documentation, traduction, ou “easy pickings”
- **Alternative** : pour une toute première contribution, un projet plus petit peut être moins intimidant

Exercice 2 — Explorer la structure d’un projet (15 min)

Fichiers à trouver (exemple Django)

Fichier	Présent ?	Contenu principal
README.md	Oui	Description, installation, liens documentation
LICENSE	Oui	BSD 3-Clause
CONTRIBUTING.md	Oui	Processus de contribution détaillé
CODE_OF_CONDUCT.md	Oui	Contributor Covenant v2.0
CHANGELOG.md	Non	Utilise les release notes sur le site
.github/ISSUE_TEMPLATE/	Oui	Templates pour bugs et features
.github/PULL_REQUEST_TEMPLATE.md	Oui	Checklist pour les PRs

Questions sur CONTRIBUTING.md

1. Comment doit-on reporter un bug ?

- Vérifier que le bug n’existe pas déjà (recherche dans les issues)
- Utiliser le template d’issue “Bug report”
- Fournir : version de Django, version Python, OS
- Inclure les étapes de reproduction minimales
- Indiquer le comportement attendu vs obtenu
- Si possible, proposer un test qui échoue

2. Y a-t-il un processus de discussion avant de coder une nouvelle feature ?

Oui :

- Les nouvelles features doivent d’abord être discutées sur django-developers (mailing list)
- Un DEP (Django Enhancement Proposal) peut être nécessaire pour les changements majeurs
- Obtenir un consensus avant de commencer à coder
- Les PRs “surprise” pour des features majeures risquent d’être refusées

3. Quels sont les standards de code (linter, formatage) ?

- Style guide Django (basé sur PEP 8 avec extensions)
- Formatage : Black (depuis Django 4.x)
- Linting : flake8, isort pour les imports
- Tests obligatoires pour tout nouveau code
- Documentation requise (docstrings, docs/)

4. Y a-t-il un DCO ou CLA ?

- Pas de CLA formel
- Les contributions sont sous licence BSD (comme le projet)
- Implicitement, en contribuant, vous acceptez la licence du projet

5. Comment les commits doivent-ils être formatés ?

- Première ligne : résumé < 70 caractères
 - Référence au ticket : “Fixed #12345 – Description”
 - Corps du message expliquant le “pourquoi”
 - Un commit = un changement logique
-

Exercice 3 — Trouver des “Good First Issues” (15 min)

Tableau des issues trouvées (exemples)

Projet	Issue #	Titre	Langage	Complexité estimée
django/django	#34567	Fix typo in documentation	Python/RST	Très faible
facebook/react	#28901	Add missing prop type validation	JavaScript	Faible
microsoft/vscode	#198765	Improve error message for invalid config	TypeScript	Moyenne

Questions

1. Parmi les issues trouvées, laquelle vous semble la plus abordable ? Pourquoi ?

La correction de typo dans la documentation Django :

- Pas besoin de comprendre le code métier
- Changement isolé et vérifiable
- Processus de contribution simple
- Faible risque de casser quelque chose
- Permet de se familiariser avec le workflow sans pression

2. Que devriez-vous faire avant de commencer à travailler sur cette issue ?

1. **Vérifier la disponibilité** : lire les commentaires, voir si quelqu’un y travaille déjà
2. **Commenter** : “I’d like to work on this issue. Is it still available?”
3. **Attendre confirmation** : quelques jours si pas de réponse
4. **Lire CONTRIBUTING.md** : comprendre le processus attendu
5. **Configurer l’environnement** : fork, clone, installation des dépendances
6. **Comprendre le contexte** : où se situe le fichier, quel est le problème exact

3. Y a-t-il des commentaires sur l’issue qui donnent des indications ?

Souvent oui :

- Le mainteneur peut indiquer le fichier concerné
 - Des suggestions d’approche peuvent être données
 - Des liens vers la documentation ou code similaire
 - Parfois un mentor est assigné pour les “good first issues”
-

Exercice 4 — Rédiger un rapport de bug (15 min)

Votre rapport de bug

Titre : Division by decimal starting with 0 returns incorrect result

Description :

When dividing a number by a decimal that starts with 0 (e.g., 0.5, 0.25), AwesomeCalc returns an incorrect result. The issue appears to be related to how decimal numbers with a leading zero are parsed.

Étapes pour reproduire :

1. Install AwesomeCalc 2.3.1: `pip install awesomecalc==2.3.1`
2. Run the calculator: `awesomecalc`
3. Enter the calculation: `10 / 0.5`
4. Observe the result

Résultat attendu :

`10 / 0.5 = 20`
(Dividing by 0.5 is the same as multiplying by 2)

Résultat obtenu :

`10 / 0.5 = 0`
(or another incorrect value - specify exactly what you see)

Environnement :

- OS: Ubuntu 22.04 LTS
- Python: 3.11.2
- AwesomeCalc version: 2.3.1
- Installation method: pip
- Shell: bash 5.1.16

Informations supplémentaires :

- Le problème ne se produit pas avec `10 / 2` (résultat correct : 5)
- Le problème ne se produit pas avec `10 / 1.5` (à vérifier)
- Premiers tests suggèrent que le "0." est peut-être ignoré lors du parsing

Auto-évaluation

- [x] Un titre clair et descriptif
- [x] Des étapes de reproduction précises
- [x] Le résultat attendu vs obtenu
- [x] Les informations sur l'environnement
- [x] Un ton respectueux et professionnel

Exercice 5 — Pratique du workflow

Questions

1. Avez-vous rencontré des difficultés ? Lesquelles ?

Difficultés courantes :

- **Configuration SSH** : clé SSH non configurée pour GitHub
- **Permissions** : fork non créé sur le bon compte
- **Remote upstream** : oubli d'ajouter le remote upstream
- **Conflits** : si le fichier a été modifié entre-temps
- **Format du commit** : message mal formaté selon les conventions

Solutions :

- Suivre la documentation GitHub pour configurer SSH
- Vérifier qu'on est sur le bon fork avant de push

- Toujours ajouter upstream après le clone
- Faire `git pull upstream main` avant de créer la branche

2. Combien de temps s'est écoulé avant que votre PR soit mergée ?

Pour le projet first-contributions :

- Généralement quelques heures à quelques jours
 - Ce projet est conçu pour merger rapidement les PRs des débutants
 - Pour un vrai projet : le temps varie (heures à semaines selon l'activité)
-

Points clés à retenir

1. **Évaluation préalable** : toujours vérifier la santé d'un projet avant de contribuer
2. **Documentation** : lire CONTRIBUTING.md et CODE_OF_CONDUCT.md obligatoirement
3. **Communication** : signaler son intention de travailler sur une issue
4. **Qualité** : rapport de bug complet = résolution plus rapide
5. **Workflow** : fork → clone → branch → commit → push → PR
6. **Patience** : les mainteneurs sont souvent bénévoles

Ressources complémentaires

- **GitHub Skills** : <https://skills.github.com/> (tutoriels interactifs)
- **Open Source Guide** : <https://opensource.guide/fr/how-to-contribute/>
- **First Contributions** : <https://github.com/firstcontributions/first-contributions>
- **Good First Issues** : <https://goodfirstissue.dev>